

11주차 배포한 모델은 어떻게 낚고, 언제 다시 배워야 할까

11주차 · 모니터링

이 장의 수치는 모두 `practice/chapter11/` 의 코드를 실행해 얻은 값이다. 예시를 위해 지어낸 숫자는 없다. 인용한 값의 출처 파일은 출력 블록 첫 줄에 적었다.

다른 주차의 산출물을 가져다 쓴 곳에는 그 주차의 `practice/chapter{N}/data/output/` 경로를 함께 적었다.

- ☐ 10주차에서 모델은 컨테이너에 실려 검증을 통과하고 배포됐음
- ☐ 배포는 모델 생애의 끝이 아니라 시작임
- ☐ 코드가 한 줄도 바뀌지 않아도 세상이 바뀌면 예측은 조용히 어긋남
 - 그 어긋남은 서버 로그에 오류로 남지 않음
- ☐ 이 장은 그 어긋남을 무엇으로 재고, 신호가 났을 때 무엇을 확인한 뒤 다시 배울지 정하는 절차를 다룬다
- ☐ 이 장도 설명과 실습을 나누지 않음
- ☐ 절마다 실행할 명령이 나오고 그 아래에 실제 출력과 해석이 이어짐
- ☐ 순서대로 실행하면 이 장의 수치를 직접 생성함

1. 오늘의 질문

- ☐ 서버가 멀쩡한데 모델만 틀리기 시작하면 무엇으로 알아차릴까?
- ☐ "오늘 예측이 맞았는지"를 오늘 알 수 없는 과제에서는 무엇을 감시할까?
- ☐ 지표 두 개가 서로 다른 답을 내면 어느 쪽을 믿어야 할까?
- ☐ 드리프트(drift, 분포 변화) 경보가 났다. 바로 다시 학습해도 될까?

2. 이 장에서 실행할 것

- ☐ 배포된 모델을 감시하는 일은 건강검진 절차와 단계가 하나씩 대응함

건강검진	모델 감시
진료실에서 바로 재는 체온·혈압	시스템 지표 — 요청 단위로 즉시 나옴
며칠 뒤 나오는 혈액검사 결과	모델 성능 — 정답이 도착해야 확정됨
검사 두 종이 다른 답을 내면 둘을 함께 읽음	KS와 PSI가 엇갈리면 조합 규칙으로 판정함
수치 하나가 기준을 넘어도 바로 수술하지 않음	경보는 확인 착수 신호이지 재학습 명령이 아님
검사 기기가 고장 났으면 치료가 아니라 재검사가 먼저임	원인이 데이터 결함이면 재학습이 아니라 수리가 먼저임

표 11.1 건강검진과 모델 감시의 대응

- ☐ 마지막 두 줄이 이 장의 결론에 해당함
- 검사 결과에서 조치까지는 한 걸음이 아니라 두 걸음이고, 그사이에 원인 확인이 들어감
- ☐ 비유가 깨지는 지점이 하나 있음
- 사람은 아프면 스스로 증상을 느끼고 병원에 가지만, 모델은 틀린 답을 내면서도 200 OK 로 정상 응답함
- 그래서 감시 장치를 모델 밖에 따로 세워야 함
- 그 장치가 낸 두 지표가 서로 다른 답을 내는 장면이 5절에서 숫자로 나타남
- ☐ 실행하기 전에 네 가지를 먼저 적음
- 산출물을 먼저 열면 거기 적힌 값이 정답처럼 보여 "왜 이 값인가"를 묻지 않게 됨

표 11.2 예측표 — 실행하기 전에 채운다

예측할 것	내 예측	실제	어긋났다면 원인
같은 날 1시간 간격 비교에서 PSI가 주의 임계 0.1을 넘는가			
9일 간격 비교에서 KS 검정이 유의($p < 0.05$)하게 나오는가			
9일 간격 비교의 알림 등급은 정상·주의·경보 중 무엇인가			
9일 뒤 비교에서 40개 측정소 중 몇 곳이 최하 구간(17 미만)에 들어가는가			

- ☐ 이 장에서 실행할 코드는 둘이고 하나는 선택임

순서	실행할 파일	하는 일	절	선행 조건
1	run_chapter11.py	비교 A·B를 차례로 돌리고 보고서를 만들	4·5·6	code/requirements.txt 설치
2	code/11-1-drift-monitoring.py	기준·현재 한 쌍만 비교(기본값이 비교 B)	5	위와 같음
선택	code/11-0-collect-snapshot.py	스냅샷(snapshot)을 새로 수집	—	data.go.kr 인증키

- 1번이 2번을 두 번 호출하므로 1번만 실행해도 이 장의 산출물이 전부 생김
- Docker나 서버가 필요 없고 Python과 numpy.pandas.scipy만 있으면 됨

```
cd practice/chapter11
python3 -m venv venv && source venv/bin/activate # Windows: venv\Scripts\activate
pip install -r code/requirements.txt
```

- 선택 실습인 11-0-collect-snapshot.py 는 인증키가 없으면 실행하지 않는다
- 이 스크립트는 data/input/ 의 스냅샷 파일을 덮어쓴다. 저장소에 커밋된 세 스냅샷이 이 장의 기준선이므로, 덮어쓰면 본문의 수치와 대조할 수 없게 된다.
- 1주차에서 만든 인증키가 있고 수집 절차를 직접 보고 싶을 때만 --out 을 새 파일명으로 지정해 실행한다.

3. 감시는 두 층이다 — 살아 있는가와 옳은 답인가

- 모델 감시를 이야기하기 전에 순서를 지켜야 함
- 예측이 틀리는 것보다 먼저, 그리고 훨씬 자주, 서비스 자체가 느려지거나 멈춤
- 응답하지 않는 API 앞에서 예측 품질은 논점이 아님
- 그래서 감시의 1층은 모델과 무관한 세 질문으로 시작함
- 요청이 얼마나 걸리는가(지연)
- 얼마나 많이 오는가(처리량)
- 얼마나 실패하는가(오류율)
- 지연은 평균으로 보면 안 됨
- 4주차 실측에서 200건을 보낸 처리 지연은 평균 3.5ms인데 최대는 110.7ms였음
- 평균만 보면 "3.5ms짜리 서비스"지만 가장 오래 기다린 요청은 그 30배 넘게 기다린 셈임
- 그래서 운영 감시는 평균 대신 분위수(p50·p95·p99)를 나란히 봄
- 처리량은 급증만 신호가 아니라 급락도 신호임
- 민원 접수가 평소의 10분의 1로 떨어졌다면 민원이 줄어든 것이 아니라 상류 수집이 멈췄을 가능성을 먼저 의심함
- 오류율은 4xx와 5xx를 나눠 셈
- 10주차에서 잘못된 입력이 422로 거절된 것은 서비스의 실패가 아니라 입력 계약이 정상 작동한 결과임
- 그러므로 오류율 0이 목표가 아님
- 다만 422가 갑자기 급증하면 호출하는 쪽이 일제히 틀리기 시작했다는 뜻이므로 상류의 스키마 변경(2주차)을 의심함

```
# 출처: practice/chapter4/data/output/ch4_delivery_report.json
```

```
"latency_ms": { "count": 200, "mean": 3.5, "max": 110.7 }
```

```
# 출처: practice/chapter10/data/output/ch10_deploy_report.json
```

```
"host": { "forecast": 6.0, "predict_status": 200, "invalid_input_status": 422 },
```

```
"shadow_observation": { "x_prev_count": 9, "champion_forecast": 6.0,  
                        "challenger_forecast": 7.3636 }
```

□ 화면에 나온 값에서 읽을 것

○ 평균과 최대가 30배 넘게 차이 남 — 지연을 한 숫자로 요약하면 가장 오래 기다린 이용자가 통계에서 사라짐

○ 정상 응답 200과 거절 422가 함께 정상임 — 422는 계약에 맞지 않는 입력을 서비스가 걸러 낸 결과이므로, 감시 규칙은 "422 발생"이 아니라 "422 급증"을 봄

○ champion과 challenger가 같은 입력에 다른 답을 냄 — 전일 9건에 champion은 6.0, challenger는 7.3636을 냄. 두 값의 격차 자체가 모델 층에서 볼 수 있는 신호임

2층이 필요한 이유 — 조용한 고장

□ 문제는 이 세 지표가 전부 정상인 채로 모델이 틀리기 시작하는 상황임

○ 서버는 빠르고 오류율은 정상이고 /health 는 ok인데 예측만 세상과 어긋남

○ 1층 감시로는 이 고장이 잡히지 않으므로 "옳은 답을 내는가"를 묻는 2층을 따로 세움

□ 2층에서 가장 정확한 지표는 성능임

○ 예측과 정답을 대조해 오차를 계산하면 됨

○ 그런데 정답이 늦게 옴

– 민원 건수 예측의 정답은 다음 날이 되어야 확정되므로 오늘의 성능을 오늘 알 수 없음

– 심사 결과나 재방문 여부처럼 며칠에서 몇 주가 걸리는 과제도 흔함

□ 이 레이블 지연 때문에 모델 감시는 정답이 도착하는 순서대로 층을 이룸

○ 정답 없이 지금 볼 수 있는 것은 입력 분포와 예측 분포임

○ 정답이 도착하면 지연된 성능 평가로 확정 판정을 내림

○ 그래서 감시 설계의 첫 질문은 "어떤 지표를 볼까"가 아니라 ***이 과제의 정답은 언제 도착하는가***다

– 정답 확정에 몇 달이 걸리는 과제에서는 입력 분포가 사실상 유일하게 볼 수 있는 신호가 됨

□ 입력 분포와 예측 분포 중 어느 쪽을 볼지는 모델이 정함

○ 9주차에서 champion으로 승격된 모델은 학습 자료의 평균을 그대로 답하는 평균 예측기임

– serving_via_alias 가 전일 9건 입력에 6.0을 답했고

(practice/chapter9/data/output/ch9_experiment_report.json), 10주차의 컨테이너와 호스트도 같은 6.0을 냈음

- 입력이 무엇이든 같은 값을 내는 모델은 입력 분포가 아무리 변해도 예측 분포가 상수로 남음
- **입력에 둔감한 모델일수록 예측 분포가 아니라 입력 분포를 직접 감시해야 한다.**
- 이 장의 실습이 하는 일이 그것임

4. 무엇이 변했는지 재려면 비교창부터 맞춘다

- 모델은 학습 당시의 데이터가 담고 있던 세상을 전제로 동작함
- 그 전제가 깨지는 방식이 둘이고, 둘은 감지 재료부터 다름
- **데이터 드리프트**는 입력의 분포가 변하는 것이다
- 휴가철이 되어 지역구별 민원 건수 자체가 커지면 모델이 받는 질문의 분포가 변한 것이며, 입력과 정답의 관계는 그대로일 수도 있음
- 입력 로그만 있으면 감지되므로 정답을 기다릴 필요가 없음
- **컨셉 드리프트**는 입력과 정답의 관계가 변하는 것이다
- 새 제도가 시행되어 같은 "전일 9건"에서 기대되는 다음 날 건수가 달라지면, 질문은 같은데 정답이 변한 것임
- 관계의 변화이므로 정답을 확보해야 확인되고, 3절의 레이블 지연이 그대로 병목이 됨
- 이 비대칭에서 두 가지 함정이 나옴
- 입력 분포가 그대로여도 관계는 변할 수 있음 — 입력 감시만으로는 보이지 않는 사각지대임
- 반대로 입력 분포가 변해도 성능은 멀쩡할 수 있음 — 모델이 둔감한 방향의 변화라면 그러함
- 두 함정을 합치면 이 장에서 가장 중요한 설계 원칙이 나옴
- **드리프트 경보는 "성능이 떨어졌다"는 확정이 아니라 "확인에 착수하라"는 신호로 설계한다.**
- 6절의 판단표가 이 원칙을 절차로 옮긴 것임
- 그런데 분포를 재기 전에 먼저 정할 것이 있음 — **무엇과 무엇을 비교할 것인가**
- 실습은 서울 40개 측정소의 PM10 실측 스냅샷 세 종을 씀
- 기준은 2026-06-28 22:00, 비교 A는 1시간 뒤인 같은 날 23:00, 비교 B는 9일 뒤인 2026-07-07 22:00임
- 이 실습이 재는 것은 민원 예측 모델의 입력이 아니라 대기질 측정값의 분포임
- 정답을 기다리지 않고 입력 분포만으로 변화를 재는 절차는 같으므로 감시 장치가 어떻게 작동하는지 그대로 볼 수 있음
- 다만 여기서 나온 등급을 민원 예측 모델의 상태로 읽으면 안 됨
- 이 자료로 확인할 수 있는 것은 데이터 드리프트뿐임
- 스냅샷에는 측정값만 있고 "정답" 열이 없으므로 관계의 변화는 이 실습에서 재지 못함
- 컨셉 드리프트를 재려면 예측과 실재를 짝지은 기록이 쌓여야 하고, 그 기록이 쌓이는 속도가 곧 레이블 지연임
- 비교 B가 **같은 22:00**인 것은 우연이 아니라 설계임
- PM10은 하루 안에서도 오르내리므로 기준 22:00과 현재 14:00을 비교하면 "9일의 변화"와 "시각의 변화"가 섞임

- 그러면 값이 커져도 어느 쪽이 원인인지 말할 수 없음
- 기준 창과 비교 창을 같은 조건으로 맞추는 것이 드리프트 감시의 기본기임
- 정렬하지 않은 비교가 만드는 것은 신호가 아니라 교란임
- 실습을 실행함. 4~6절의 숫자는 전부 이 명령의 출력임

```
cd practice/chapter11
python run_chapter11.py
```

```
# 출처: practice/chapter11/results/run_chapter11.log
[기준] 2026-06-28 22:00 n=40 평균 20.025
[현재] 2026-06-28 23:00 n=40 평균 20.775
KS=0.175 p=0.5786 (alpha=0.05) | PSI=0.1193 | KL(b-c)=0.0646 KL(c-b)=0.0548
알림: KS유의=False PSI대역=주의 → 등급 주의
...
=== 비교 A: 22:00 vs 23:00 (6/28, 2장 재현 검증) ===
2장 실측 재현: True
...
CH11_RUN_PASS
```

- 화면에 나온 값에서 읽을 것
- **2장 실측 재현: True 가 먼저다** — 같은 스냅샷 쌍에서 2주차가 얻은 KS 0.175와 p 0.5786이 그대로 다시 나왔음(practice/chapter2/data/output/ch2_drift_result.json)
- 새 지표를 엮기 전에 옛 지표가 재현되는지부터 확인해야 값이 달라졌을 때 원인이 자료인지 코드인지 가를 수 있음
- **1시간 간격에도 PSI가 0.1193으로 주의 대역에 들어감** — 2주차가 "값은 변해도 구조는 같다"로 판정한 쌍인데 PSI는 관행 임계 0.1을 넘겼음
- **마지막 줄이 CH11_RUN_PASS 임** — 두 비교의 스냅샷 시각, 구간 수, 등급 값이 모두 기대와 맞았다는 뜻임. 이 줄이 없으면 아래 수치를 읽지 않음

- **1시간 뒤의 "주의"는 대기질이 나빠졌다는 뜻이 아니다**
- 5구간에 40건이면 구간당 기대가 8건이므로, 측정소 두세 곳이 구간 경계를 넘나들기만 해도 비율이 몇 %p씩 흔들린다.
- 관행 임계를 소표본에 그대로 적용하면 평시에도 매시간 주의가 발생한다는 것을 실데이터가 보여 준 셈이다.

5. KS와 PSI는 다른 질문에 답한다

- ☐ 입력 분포가 변했는지 재는 도구는 하나가 아님
- 이 장은 둘을 함께 쓰는데, 성격이 달라서 하나를 고르는 것이 아니라 둘 다 계산한 뒤 조합해 판정함
- ☐ **KS 2표본 검정**은 2주차에서 이미 다뤘음
- 두 표본의 누적분포 최대 간격(통계량 D)을 재고, "같은 분포에서 나왔다"는 가정 아래 그 간격이 우연히 나올 확률(p-value)을 돌려줌
- 2주차의 결론을 그대로 가져옴
- p-value가 크다는 것은 "분포가 같다"가 아니라 "다르다고 볼 증거가 부족하다"는 뜻임
- 그리고 결과가 표본 크기에 좌우됨
- ☐ ****PSI(Population Stability Index, 모집단 안정성 지수)****는 이 장에서 새로 배우는 지표임
- 기준 분포를 분위수로 잘라 구간을 만들고, 구간마다 비율이 얼마나 이동했는지를 더함

$$PSI = \sum (현재비율 - 기준비율) \times \ln(현재비율 / 기준비율)$$

- 두 항의 부호가 곱해져 언제나 양수가 되므로, 비율이 늘어난 구간과 줄어든 구간이 모두 값을 키움
- 신용평가 업계에서 모집단이 흔들렸는지 점검하려고 쓰던 지표임
- 관행 임계는 0.1을 주의, 0.25를 경보로 봄
- 이 임계에 통계적 근거가 약하다는 지적이 있음(Yurdakul 2018). **관행 임계는 출발점이지 법칙이 아니다**
- ☐ PSI를 계산하려면 먼저 구간을 만들어야 하고, 그 구간의 자(尺)는 기준 분포가 줌
- 기준 표본의 분위수로 경계를 잘라 다섯 구간을 만듦
- 40건을 다섯으로 나누므로 기준에서 구간마다 8건 안팎이 들어감
- 양 끝 구간은 위아래로 열어 둬
- 현재 값이 기준의 범위를 벗어나도 버려지지 않고 끝 구간에 잡힘. 범위 밖 값이야말로 변화의 신호이기 때문임
- 어느 구간의 비율이 0이 되면 로그가 발산하므로 구간마다 0.5를 더해 계산함
- 그래서 기준의 첫 구간 7개소가 $7 \div 40 = 0.175$ 가 아니라 $(7+0.5) \div 42.5 = 0.1765$ 로 기록됨
- (`practice/chapter11/data/output/ch11_compare_b_9days.json`의 `baseline_counts` · `baseline_props` 첫 값)
- 구간 수가 달라지면 PSI 값끼리 비교할 수 없으므로, 코드는 분위수가 겹쳐 구간이 줄면 경고를 냄
- ☐ 두 지표는 묻는 것이 다름
- KS는 "이 차이를 우연으로 보기 어려운가"에 답하고, PSI는 "이동의 크기가 얼마인가"에 답함
- 질문이 다르므로 답이 엇갈릴 수 있고, 실제로 비교 B에서 엇갈림
- ☐ 표본이 크면 사소한 차이도 KS에서 전부 "유의"로 나오므로 실질 판단은 거리 지표인 PSI가 맞음
- ☐ 반대로 표본이 작으면 PSI가 몇 건의 이동에도 출렁이므로 이번에는 KS가 제동 장치가 됨
- 두 지표를 함께 요구하는 규칙은 이 두 방향의 오판을 한 번에 막으려는 설계다.
- ☐ 비교 B만 따로 실행함. 기본값이 9일 간격 쌍이므로 인자가 없어도 같은 비교가 나옴

```
cd practice/chapter11
python code/11-1-drift-monitoring.py
```

```
# 출처: practice/chapter11/results/11-1-drift-monitoring.log
[기준] 2026-06-28 22:00 n=40 평균 20.025
[현재] 2026-07-07 22:00 n=40 평균 17.75
KS=0.225 p=0.2657 (alpha=0.05) | PSI=0.3324 | KL(b→c)=0.1826 KL(c→b)=0.1497
알림: KS유의=False PSI대역=경보 → 등급 주의
권고: 모니터링 강화 + 원인 분류 착수(계절/정책/시스템 변경/데이터 결함)
```

```
# 출처: practice/chapter11/data/output/ch11_compare_b_9days.json
"labels": ["[-inf, 17.0)", "[17.0, 19.0)", "[19.0, 21.0)", "[21.0, 22.0)", "[22.0, inf)"],
"baseline_counts": [7, 5, 10, 5, 13],
"current_counts": [13, 7, 11, 5, 4]
```

□ 화면에 나온 값에서 읽을 것

○ 분포 전체가 아래로 이동했음 — 최하 구간(17 미만)이 7개소에서 13개소로 늘고 최상 구간(22 이상)이 13개소에서 4개소로 줄었으며, 평균은 20.025에서 17.75로 내려갔음

○ 두 지표가 다른 답을 냄 — 같은 이동을 PSI는 0.3324로 "경보 대역"이라 하고, KS는 $D=0.225$ 에 $p=0.2657$ 로 "유의하지 않다"고 함

– $p=0.2657$ 은 유의수준 0.05보다 크므로 이 표본 크기에서는 우연을 배제하지 못한다는 뜻임

○ 한 문장으로 옮기면 이렇게 됨 — "이동의 크기는 경보 수준인데, 이 표본 크기에서는 우연을 배제할 수 없다"

○ 간격이 늘자 세 지표가 모두 같은 방향으로 커졌음 — 1시간에서 9일로 가며 KS는 $0.175 \rightarrow 0.225$, PSI는 $0.1193 \rightarrow 0.3324$, KL(기준→현재)은 $0.0646 \rightarrow 0.1826$ 이 됨

□ 엇갈린 두 답을 하나의 등급으로 확정하는 것이 조합 규칙임

```
# practice/chapter11/code/11-1-drift-monitoring.py 발췌
if ks_significant and psi >= PSI_ALERT:      #  $p < 0.05$  그리고  $PSI \geq 0.25$ 
    level = "경보"
elif ks_significant or psi >= PSI_WATCH:     # 둘 중 하나만
    level = "주의"
else:
    level = "정상"

# 비교 B 실측: ks_significant=False, psi=0.3324
# 첫 조건 → False and True → 거짓
# 둘째 조건 → False or True → 참 ⇒ level = "주의"
# 출처: practice/chapter11/results/11-1-drift-monitoring.log
# "알림: KS유의=False PSI대역=경보 → 등급 주의"
```

- 경보에 KS 유의와 PSI 0.25를 **AND**로 묶은 것이 핵심임
- OR로 묶었다면 둘 중 하나만 걸려도 경보가 되어 사람을 부르게 되고, 소표본에서 PSI가 흔들릴 때마다 헛경보가 나감
- 헛경보를 서른 번 겪은 담당자는 서른한 번째를 확인하지 않으므로, 사람 호출은 가장 높은 등급에만 불임
- 비교 B는 PSI만 경보 대역이었으므로 규칙이 한 단계 낮은 주의로 확정했고, 그 판정이 곧 "사람을 부르지 않고 기록을 쌓는다"는 뜻임

6. 경보는 명령이 아니다 — 재학습 판단

- 경보가 났다고 재학습 버튼부터 누르면 안 됨
- 알림과 조치 사이에 두 개의 확인이 들어감
- 첫째는 **원인 분류**다
 - 2주차에서 분포 변화의 원인을 정책 변화·계절성·외부 이벤트·시스템 변경으로 나눴는데, 운영에서는 하나가 더 붙음
 - 상류 파이프라인이 깨져 단위가 바뀌었거나 결측이 급증한 **데이터 결함**임
 - 이때 필요한 조치는 재학습이 아니라 수리임. 손상된 데이터로 다시 배우면 손상된 상태를 학습한 모델이 나옴
 - 원인이 계절 재발이라면 임계 보정이나 계절 피쳐 추가가 답임
 - 매년 여름의 같은 변화를 매년 "이상"으로 경보하면 알림 피로만 쌓임
 - 정책 시행 같은 진짜 세상의 변화일 때에만 재학습이 후보가 됨
- 둘째는 **성능 확인**이다
 - 4절에서 봤듯 입력 분포의 변화가 성능 저하를 보장하지 않음
 - 정답이 도착하는 과제라면 지연된 성능 평가로 저하를 확인한 뒤 움직임

- 두 확인이 실제로 어떻게 흐르는지 월요일 아침 상황으로 봄
- 대시보드에 입력 분포 경보가 떴다면 당직자는 먼저 세 곳을 확인함
 - 상류 수집 로그에서 주말 배치가 정상 종료했는지 봄(데이터 결함 여부)
 - 달력에서 직전이 연휴였는지 봄(계절 재발 여부)
 - 정책 공지에서 새 제도가 그 주에 시행됐는지 봄
- 성능 확인 측은 이 아침에 비어 있음. 어제 실제 건수는 오늘 밤에야 확정되기 때문임
- 그래서 아침의 조치는 재학습 결재가 아니라 원인 분류 착수와 기록 축적에 머물
- 실습이 만든 보고서에는 판정에 쓴 규칙과 권고가 지표와 분리되어 기록됨

```
# 출처: practice/chapter11/data/output/ch11_drift_report.json
"alert_rule": {
  "ks": "p < 0.05 (2장과 동일 alpha)",
  "psi": "0.1 주의 / 0.25 경보 (신용평가 업계 관행 - Yurdakul 2018)",
  "combine": "경보=KS유의 AND PSI>=0.25, 주의=둘 중 하나, 정상=둘 다 미달" },
"comparison_a_matches_ch2": true,
"notes": [
  "비교창 정렬: 기준·현재 모두 22:00 정시 - 일중 변동 교란을 설계로 제거",
  "n=40 소표본 - KS 검정력 낮음, PSI 구간당 기대 8건으로 출렁임에 민감",
  "알림≠재학습: 권고 조치는 원인 분류를 먼저 요구(11.4)" ]
```

- 화면에 나온 값에서 읽을 것
- 판정 규칙이 산출물 안에 글자로 남음 — 나중에 등급을 두고 다투면 그때 쓰인 임계와 조합 방식을 파일에서 확인할 수 있음
- 권고가 "재학습"이 아니라 "원인 분류 착수"임 — 지표·등급·조치를 각각 다른 필드에 적어 둔 설계가 그 분리를 강제함
- 소표본이라는 한계가 보고서에 함께 적힘 — 유리한 해석만 남기지 않고 판정의 약점을 같은 파일에 적는 것이 감사 대비의 기본임

표 11.3 재학습 트리거 판단표

드리프트 경보	성능 저하(정답 확인)	원인 분류	조치
정상	—	—	기준선 유지, 정기 감시
주의	미확인(정답 대기)	착수	감시 강화, 기록 축적 — 사람 호출 없음
경보	확인됨	데이터 결함	수리(재학습 금지 — 깨진 세상을 학습하게 됨)
경보	확인됨	계절 재발	임계 보정·계절 피쳐 검토
경보	확인됨	실제 변화(정책 등)	재학습 → challenger 등록 → 9주차 게이트
경보	저하 없음	실제 변화	감시 지속 — 모델이 둔감한 방향의 변화

- 재학습을 언제 돌릴지 정하는 방식은 셋임
- 정기 트리거는 주기마다 무조건 재학습하므로 단순하지만 변화가 없어도 비용이 듦
- 이벤트 트리거는 판단표가 경보를 확정할 때만 움직이므로 효율적이지만 감시 장치가 선행 투자임
- 실무의 기본값은 둘을 섞어, 정기 재학습으로 바닥을 깔고 경보가 뜨면 앞당겨 실행하는 방식임
- 어느 경로든 착지점은 같음
- 재학습된 모델은 champion을 곧바로 교체하지 않고 **challenger로 등록되어 9주차의 승격 게이트를 다시 통과해야 하며**, 통과하면 10주차의 배포 절차를 다시 밟음
- 급하다고 게이트를 건너뛰면 9주차에서 세운 감사 가능성이 그 지점에서 꺾임
- 데이터(4~7주차) → 훈련·등록(9주차) → 배포(10주차) → 감시(11주차) → 재학습 → 다시 9주차로 돌아오는 이 순환이 MLOps를 파이프라인이 아니라 루프라고 부르는 이유다

7. 핵심 요약

- **1. 감시는 두 층이고, 2층이 없으면 조용한 고장을 놓친다.**
 - 지연·처리량·오류율은 "서비스가 살아 있는가"에만 답하므로, 서버가 멀쩡한 채 예측만 틀리는 고장은 여기에 잡히지 않음
 - 지연은 평균 3.5ms와 최대 110.7ms가 한 실행 안에 함께 나타났으므로 분위수로 봄
 - 422 거절은 계약이 작동한 결과이고, 감시해야 할 것은 발생이 아니라 급증임(3절)
- **2. 정답이 언제 도착하는가가 1차 신호를 정한다.**
 - 민원 건수 예측의 정답은 다음 날 확정되므로 오늘의 성능은 오늘 알 수 없고, 그동안 볼 수 있는 것은 입력 분포와 예측 분포임
 - 9주차 champion은 전일 9건에도 6.0을 답하는 평균 예측기라 예측 분포가 상수로 남으므로, 이런 모델은 입력 분포를 직접 감시함(3절)
- **3. 비교창을 맞추지 않으면 나온 값이 신호가 아니라 교란이다.**
 - 비교 B를 기준과 같은 22:00으로 고정한 것은 일중 변동을 설계로 제거하려는 조치임
 - 같은 스냅샷 쌍에서 2주차 실측(KS 0.175, p 0.5786)이 재현됐음을 먼저 확인한 뒤에 새 지표를 읽음(4절)
- **4. KS와 PSI는 다른 질문에 답하므로 함께 요구한다.**
 - KS는 "우연인가", PSI는 "얼마나 큰가"에 답함
 - 9일 비교에서 PSI 0.3324는 경보 대역인데 KS는 $p=0.2657$ 로 유의하지 않았고, AND 규칙이 이를 주의로 확정했음
 - OR로 묶었다면 소표본이 흔들릴 때마다 사람이 호출됐을 것임(5절)
- **5. 경보와 재학습 사이에는 두 개의 확인이 있다.**
 - 원인이 데이터 결함이면 재학습이 아니라 수리이고, 계절 재발이면 임계 보정임
 - 정답으로 성능 저하까지 확인된 실제 변화일 때에만 재학습이 후보가 됨
 - 재학습된 모델도 challenger로 9주차 게이트를 다시 통과함(6절)

8. 과제

☐ 실습을 실행해 만든 `practice/chapter11/data/output/ch11_drift_report.json` 을 LMS에 업로드한다

☐ 이어서 세 가지를 각각 세 문장 이내로 쓴다

1. 본인 산출물의 두 비교에서 `ks_significant` 와 `psi_band` 가 어떻게 엇갈렸는지 값으로 적고, 그럼에도 `alert_level` 이 그렇게 정해진 과정을 `alert_rule` 의 `combine` 규칙으로 재현한다.
2. `combine` 규칙이 KS와 PSI를 AND로 묶은 이유를 쓰고, OR로 묶었다면 운영에서 무엇이 달라졌을지 쓴다.
3. 드리프트 경보와 성능 저하가 함께 확인됐는데 원인이 데이터 결함으로 밝혀졌다면 왜 재학습을 실행하면 안 되는지, 그때 어떤 모델이 나오는지로 설명한다.

☐ 수업일 포함 7일 이내에 제출한다

9. 더 알아보기

☐ 이 장에서 자른 내용은 아래에서 이어 볼 수 있음

○ **PSI와 KL divergence의 관계** — 산출물의 `psi_equals_kl_sum` 이 `true` 인 것은 PSI가 양방향 KL의 합과 같다는 항등식을 수치로 확인한 결과임. 구간 확률과 스무딩 처리는 `practice/chapter11/code/11-1-drift-monitoring.py` 의 `binned_proportions.psi_and_kl` 함수에 있음

○ **구간 수를 바꿔 보기** — `11-1-drift-monitoring.py` 에 `--bins 10` 을 주면 같은 자료에서 PSI가 어떻게 달라지는지 볼 수 있음. 구간 수가 다르면 PSI끼리 비교할 수 없다는 경고가 코드에 들어 있음

○ **스냅샷 수집 절차** — 정시 데이터가 정시보다 늦게 게시되는 문제를 풀러로 다루는 방법:

`practice/chapter11/code/11-0-collect-snapshot.py` 와 `practice/chapter11/data/output/ch11_collect_log.txt`

○ **골든 시그널의 원문** — 지연·트래픽·오류·포화 네 가지의 정의: <https://sre.google/sre-book/monitoring-distributed-systems/>

○ **KS 2표본 검정의 계산 규약** — https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.ks_2samp.html

○ **피처가 바뀔 때의 감시** — 입력 자체의 정의가 바뀌는 경우의 관리는 12주차, 감시까지 포함한 통합 사례는 13주차, 생성형 모델의 감시는 14주차에서 다룸